

CURSOR DEVELOPMENT GUIDE

Trustworthy AI for Protein Engineering

Discovering Mutation-Space Reliability and Applying It to Industrial Carbon-Capture Enzyme Design

Complete Development Framework, Engineering Standards, AI-Assisted Coding Methodology, Validation Strategy, Scientific Traceability System, And Production Build Guide

Version 2.0

Purpose

This document defines how Cursor should be used to develop the entire project.

The purpose is not merely to generate code.

The purpose is to ensure Cursor develops a complete scientific platform that is:

Scientifically Correct

Architecturally Consistent

Reproducible

Testable

Traceable

Maintainable

throughout the entire lifecycle of the project.

Development Philosophy

Cursor should not be treated as:

A Code Generator

Cursor should be treated as:

Lead Software Engineering Partner

responsible for implementing a scientific research platform according to:

FINAL PROJECT BLUEPRINT

MASTER IMPLEMENTATION SPECIFICATION

SCIENTIFIC ALGORITHMS SPECIFICATION

DATASET SPECIFICATION

REPOSITORY ARCHITECTURE

The architecture documents are the source of truth.

Cursor must implement those specifications.

Cursor must never redefine them.

The Golden Rule

Before generating any code, Cursor must understand:

Scientific Requirement

↓

Algorithm

↓

Dataset

↓

Code

↓

Validation

Code without scientific traceability is unacceptable.

Primary Development Objective

The objective is to create a system capable of answering:

Can mutation-space sparsity
predict AI prediction failure?

and transforming that discovery into:

Reliability Estimation

,

and ultimately:

Reliability-Guided
Protein Engineering

Official Development Sequence

The project must be developed in the following order.

No stage may be skipped.

Stage 1

Infrastructure

Objective

Create the foundational platform.

Required components:

Configuration Layer

Logging System

Experiment Tracking

Dataset Registry

Checkpoint Management

Cursor Deliverables

src/infrastructure/

tests/infrastructure/

Validation Requirement

Before moving forward:

Unit Tests

PASS

Configuration Tests

PASS

Logging Tests

PASS

Stage 2

Dataset Layer

Objective

Build dataset ingestion and validation systems.

Required capabilities:

Data Loading

Schema Validation

Dataset Registration

Version Tracking

Cursor Deliverables

src/datasets/

tests/datasets/

Required Tests

Schema Tests

Integrity Tests

Missing-Value Tests

Stage 3

Objective

Implement empirical sparsity calculations.

Reference:

SCIENTIFIC ALGORITHMS
SPECIFICATION

Cursor Deliverables

src/sparsity/empirical/
..

Outputs:

empirical_sparsity.parquet

Stage 4

Evolutionary Sparsity

Objective

Implement ESM-based evolutionary sparsity.

Cursor Responsibilities

Build:

Model Loader

Mutation Scoring

Probability Extraction

Normalization Logic

Deliverables

src/sparsity/evolutionary/

Outputs:

evolutionary_sparsity.parquet

Stage 5

Structural Sparsity

Objective

Implement structure-based sparsity.

Cursor Responsibilities

Build:

SASA Calculations

Residue Exposure

Normalization

Outputs:

structural_sparsity.parquet
``

Stage 6

Combined Sparsity

Objective

Combine all sparsity metrics.

Outputs:

combined_sparsity.parquet

Validation Requirement

Cursor must implement:

Range Checks

Normalization Checks

Distribution Checks

Stage 7

Experiment 1

Antisymmetry Analysis

Objective

Determine whether sparsity predicts thermodynamic inconsistency.

Cursor Responsibilities

Build:

Forward Prediction Engine

Reverse Prediction Engine

Antisymmetry Calculator

Statistical Analysis

``

Deliverables

src/experiment1/

Outputs:

antisymmetry_results.parquet

`

Stage 8

Experiment 2

Epistasis Analysis

Objective

Determine whether sparsity predicts prediction failure.

Cursor Responsibilities

Build:

Experimental Loader

Prediction Engine

Epistasis Calculator

Error Quantification

Outputs:

epistasis_results.parquet

Stage 9

Integrated Reliability Analysis

Objective

Unify both experimental pathways.

Cursor Responsibilities

Build:

Integrated Dataset Builder

Correlation Analysis

Regression Analysis

Reliability Pathway Analysis

Outputs:

integrated_reliability_analysis.parquet

Stage 10

Reliability Framework

Objective

Transform sparsity into confidence estimation.

Cursor Responsibilities

Build:

Reliability Score Generator

Reliability Classifier

Validation Engine

Outputs:

reliability_scores.parquet

Stage 11

Transketolase Landscape

Objective

Generate the complete TKT mutation space.

Cursor Responsibilities

Build:

Mutation Enumerator

Landscape Generator

Mutation Registry

Outputs:

tkt_single_mutation_landscape.parquet

Stage 12

TKT Analysis

Objective

Apply reliability framework across all TKT mutations.

Cursor Responsibilities

Generate:

Sparsity

Reliability

Predicted Stability

Outputs:

tkt_mutation_analysis.parquet

Stage 13

Active-Site Analysis

Objective

Control for mechanistic explanations.

Cursor Responsibilities

Build:

Active-Site Mapper

Distance Calculator

Control Analysis

Outputs:

active_site_analysis.parquet

Stage 14

Industrial Fitness Framework

Objective

Rank mutations by engineering usefulness.

Cursor Responsibilities

Build:

Fitness Scoring Engine

Ranking Engine

Candidate Generator

Outputs:

industrial_fitness_scores.parquet

top_candidate_mutations.csv

Stage 15

Publication Layer

Objective

Automatically generate publication figures.

Cursor Responsibilities

Build:

Figure Generators

Statistics Tables

Supplementary Assets

Deliverables:

src/visualization/

src/publication/

Required Figures

F1 Architecture

F2 Sparsity vs Antisymmetry

F3 Sparsity vs Epistasis

F4 Antisymmetry vs Epistasis

F5 Reliability Distribution

F6 TKT Reliability Landscape

F7 Industrial Fitness Landscape

F8 Top Candidate Mutations

F9 Killer Figure

Stage 16

Competition Layer

Objective

Generate competition materials.

Cursor Responsibilities

Generate:

Science Fair Graphics

Judge Handouts

Executive Summary Assets

Presentation Graphics

Stage 17

Release Layer

Objective

Package the complete project.

Cursor Responsibilities

Generate:

Dataset Packages

Source Packages

Documentation Packages

Archive Packages

Development Standards

Rule 1

Single Responsibility

Every module should perform:

One Primary Task

Rule 2

No Hidden Dependencies

Imports should be explicit.

Rule 3

Configuration Driven

Avoid hardcoded:

Paths

Thresholds

Parameters

Rule 4

Reproducibility First

Every execution must be reproducible.

Rule 5

Validation First

Every module requires:

Input Validation

Output Validation

Failure Detection

Required Test Structure

For every module create:

Unit Tests

Integration Tests

Validation Tests

Failure Injection Tests

Example

src/reliability/

tests/reliability/

test_unit.py

test_integration.py

test_validation.py

test_failure_modes.py

Scientific Traceability Requirements

Every module must map to:

Requirement

↓

Algorithm

↓

Dataset

↓

Code

↓

Test

Required Header Template

Every major module should document:

Scientific Purpose

Inputs

Outputs

Algorithm Reference

Dataset Reference

Validation Reference

Cursor Prompt Strategy

All development prompts should follow:

1

Goal

2

Scientific Requirement

3

Input Dataset

4

Expected Outputs

5

Validation Requirements

6

Required Tests

This dramatically improves implementation quality.

Validation-First Development

Cursor must never consider a module complete because:

Code Executes

A module becomes complete only when:

Code Executes

Tests Pass

Validation Passes

Outputs Match Specification

Certification Workflow

Every phase concludes with:

Implementation Certification

Dataset Certification

Validation Certification

AI-Assisted Development Rules

Cursor should:

Generate Complete Modules

Generate Complete Tests

Generate Documentation

Generate Validation Logic

Cursor should not:

Leave TODO Blocks

Leave Incomplete Functions

Leave Undefined Interfaces

Failure Recovery Strategy

If a generated module fails:

Identify Requirement



Identify Algorithm



Identify Dataset



Identify Failure



Correct Implementation

Never patch code without tracing the scientific requirement.

Scientific Positioning

Throughout development Cursor must remember:

The project is not primarily:

Protein Stability Prediction

The project is not primarily:

Mutation Ranking

The project is primarily:

Trustworthy AI Research

focused on:

Prediction Reliability

with:

Protein Engineering

serving as the experimental domain.

This positioning should influence:

Architecture

Naming

Documentation

Figures

Publications

Competition Materials

Project Completion Definition

Cursor development is complete only when:

Infrastructure Operational



Dataset Framework Operational



Empirical Sparsity Operational



Evolutionary Sparsity Operational



Structural Sparsity Operational



Combined Sparsity Operational



Experiment 1 Operational



Experiment 2 Operational



Integrated Analysis Operational



Reliability Framework Operational



TKT Analysis Operational



Active-Site Analysis Operational



Industrial Fitness Framework Operational



Publication Assets Operational



Competition Assets Operational



Release Package Operational



and all outputs are reproducible and validated.

Final Development Statement

Cursor is responsible for implementing a complete scientific platform capable of executing the end-to-end pathway:

Mutation Space



Empirical Sparsity

Evolutionary Sparsity

Structural Sparsity



Combined Sparsity



Prediction Failure Analysis



Reliability Framework



Transketolase Engineering



Industrial Fitness Framework



Publication Assets



Competition Assets



Project Release

in a reproducible, scientifically defensible, and publication-ready manner.

END OF CURSOR DEVELOPMENT GUIDE

Version 2.0

Approved Development Methodology For The Trustworthy AI For Protein Engineering Project.